

UT 07 – Ejercicios

Ejercicio 12 – Interfaces.

Declara una interfaz Vehiculo, con las siguientes características:

- Atributo público:
 - “VELOCIDAD_MAXIMA”, con valor 120.
- Métodos abstractos:
 - void frenar (int cuanto)
 - void acelerar (int cuanto)
 - int getNumPlazas()

A continuación, crea dos clases “Coche” y “Moto” que implementen la interfaz anterior, sin utilizar ninguna otra clase / interfaz, además de las indicadas.

Ten en cuenta que:

- Puedes crear los atributos o métodos privados que necesites, así como sobrescribir los métodos abstractos que se requieran.
- Los métodos frenar y acelerar deben impedir que el vehículo decelere por debajo de 0, y que sobrepase 120. También deben mostrar un mensaje en consola indicando qué están haciendo, y qué tipo de vehículo es. Algo así como “Soy un coche. Acelero hasta ...” o “Soy una moto. No puedo acelerar más”.

Crea un programa para probar estas dos clases y el interfaz. Crea dos variables de tipo “Vehiculo”, y asigna a cada una de ellas un objeto Coche y otro Moto, respectivamente. Llama a los métodos para probarlos.

Ejercicio 13 – Interfaces + clases abstractas

Haz una copia del ejercicio anterior, y modifícalo creando una clase abstracta “VehiculoMotor” que implemente el interfaz vehículo, y de la que hereden las clases Coche y Moto.

Implementa en esta clase abstracta todo aquello que consideres común a las clases Moto y Coche, para así reducir la base de código que se debe mantener.

En concreto, estudia la forma en la que podemos hacer llamadas a métodos de la interfaz, sobrescritos en la clase abstracta, y que volvemos a sobrescribir en las clases coche y moto. Por ejemplo, acelerar, en la clase coche, puede llamar al método de la clase abstracta, después de indicar qué tipo de vehículo es.

Ejercicio 14 – Clases de utilidad – Ordenación de arrays de tipos simples.

En Java existe una clase “Arrays” (java.util.Arrays) que proporciona métodos estáticos para poder ordenar arrays de distintos tipos de datos. En concreto el método “sort” tiene varias sobrecargas para ordenar diferentes tipos de datos.

Investiga este método, y realiza un programa que:

- Cree un array de tipos primitivos (int, double, byte, etc., elige el que quieras) y lo rellene con datos aleatorios.
- Muestre el array desordenado
- Ordene el array usando el método sort de la clase Arrays.
- Muestre de nuevo el array, que debe estar ordenado.

Ejercicio 15 – Clases de utilidad - Ordenación de arrays de tipos complejos (Strings).

Al igual que ordena arrays de tipos simples, `Arrays.sort` puede ordenar arrays de objetos.

Para probarlo, vamos a usar el tipo de objeto “String”. Para comprobar que funciona, crea un programa que:

- Cree un array de cadenas y se les asigne un valor aleatorio. Conseguir que las cadenas estén formadas sólo por letras y números sería lo ideal.
- Muestre las cadenas generadas aleatoriamente.
- Ordene el array de cadenas
- Muestre el array ya ordenado

Ejercicio 16 – Clases de utilidad – Interfaces – Ordenación de arrays de tipos complejos (custom).

Vamos a ver si podemos ordenar tan fácilmente un array de clases creadas por nosotros.

Para probar, vamos a definir una clase “Persona” con los atributos “nombre” y “apellidos”, y un método `toString` que devuelve el nombre y los apellidos.

Crea, usando la clase anterior, un programa que cree un array de personas, y llénalo de objetos con nombres y apellidos diferentes.

Intenta ordenar el array usando `Arrays.sort`, y veamos qué ocurre.

Busca información de cómo solucionar los problemas que puedan surgir.

Pista: puede que tengamos que implementar una interfaz.